SurviveCity

An OpenEnv-Compliant Multi-Agent Environment for Cross-Episode Failure-Replay Learning in LLMs

Sirjan Singh[1] and Eeshan Singh[1]

[1]Team PyGuys — Meta × PyTorch × Scaler OpenEnv Hackathon, India 2026

April 2026

SurviveCity in five numbers

12	GRPO steps in 3 h 53 min on a single Colab T4
100 %	valid JSON across the trained eval (zero parse fails)
2.0×	baseline episode length (37.6 vs. 19.1 steps)
1.7×	baseline mean reward (0.80 vs. 0.46)
0 % → 10 %	survival rate; one n_t=10 episode hit the 100-step horizon (reward 1.97)

An extended 4000-step run on Kaggle gets survival to 12 % and shows near-certain (∼1.0) detection of the hidden infected agent by t≈80 — see §5.5, Fig. 5.

1 Why this env exists

Most LLM-agent benchmarks test single-agent goal completion in static environments [3]. Two phenomena central to multi-agent reasoning are not directly probed: **cross-episode learning from failure** and **hidden-role theory of mind**. SurviveCity targets both in one env.

Two new things this submission gets to test:

- **Post-mortem replay.** Each death emits a deterministic post-mortem string. The next episode prepends the agent's last N=3 post-mortems to its system prompt — a within-training cross-episode learning loop *without* an external memory module or LLM-as-judge.
- **Hidden-role social deduction.** Exactly one of three agents is silently infected at t=0. Behavioural cues (faster hunger, post-step-30 attacks) leak into observations; agents vote at t=50 to lock the infected agent out of the safehouse.

KEY TAKEAWAY. The env is a vehicle for testing whether RL on a small open-source LLM can learn (a) to read others through behaviour and (b) to learn from its own past failures — both without adding external memory or fuzzy judges.

Mapping to rubric. *Innovation (40%)*: post-mortem replay. *Pipeline (10%)*: env design + reward composition. *Reward improvement (20%)*: GRPO training run + step-12 eval. *Storytelling (30%)*: this writeup + demo video + live Space.

2 The environment, in one page

2.1 World and agents

A fixed 10×10 grid (Figure 1) with four food depots (`F`) near the corners, a 3×3 central safehouse (`S`) at rows 4–6 cols 4–6 that heals 1 HP per occupied step, and eight wall cells creating chokepoints. Three agents (A0, A1, A2) start inside the safehouse with `hp`=3, `hunger`=0. Three zombies (Z0, Z1, Z2) spawn at three of the four grid corners, move 1 cell per step toward the nearest non-safehouse agent via BFS, and cannot enter safehouse cells. Hunger increments by 1 per agent action; `hunger` ≥ 15 deals 1 HP per step. Eating on a food cell resets hunger.

```
Z . . . . . . . . Z
. F . . . . . . F .
. . . . . # . . . .
. . . # . . # . . .
. . . . S S S . . .
. . # . S S S # . .
. . . . S S S . . .
. . . # . # # . . .
. F . . . . . . F .
. . . . . . . . . Z
```

Figure 1: Fixed 10×10 layout. `F` = food (4, at the inner corners), `S` = 3×3 central safehouse, `#` = wall, `Z` = zombie spawn (3 zombies, one per occupied corner). Agents start inside `S`.

2.2 Phased mechanics

Phase	Steps	Mechanic
Pre-reveal	1–29	Survival. Infected agent's hunger increments at 1.5× rate.
Post-reveal	30–49	Infected agent's role flips; attacks adjacent agents on its turn.
Vote	50	All living agents cast `vote_lockout(target_id)`. Majority locks out one agent.
Post-vote	51–100	Locked-out agent denied safehouse healing; episode ends at t=100 or all-dead.

Table 1: Episode phases. The single-decision vote at t=50 tests whether the policy can integrate ∼50 steps of behavioural evidence into one categorical choice.

2.3 Action space

```python
class SurviveAction(BaseModel):
    agent_id:    int
    action_type: Literal["move_up", "move_down", "move_left", "move_right
        ",
                         "eat", "wait", "vote_lockout", "broadcast"]
    vote_target: Optional[int] = None    # required for vote_lockout
    message:     Optional[str] = Field(default=None, max_length=40)
```

The 40-character cap on `message` is deliberate: it forces the policy to emit terse, demonstrable theory-of-mind communication if it learns to broadcast at all.

2.4 Cross-episode post-mortems

When an agent dies the env emits a deterministic string with cause, position, nearest threat, food consumed, final hunger, and a rule-based mistake label. Sample:

```
POSTMORTEM for A1: died at step 38 (cause: zombie_attack).
Last position: (6,1). Nearest threat at death: zombie at (6,2), dist=1.
Resources consumed: 2 food. Final hunger: 7.
Key mistake: foraged_too_far_from_safehouse.
```

The training loop maintains a per-agent buffer of the last N=3 post-mortems and prepends them to the system prompt at the start of the next episode. **This is the cross-episode failure-replay mechanism**: the policy is given concrete textual evidence of past failure modes without any LLM-as-judge or external memory module. The text is rule-based and deterministic, satisfying the OpenEnv validator's no-fuzziness requirement.

3 Reward, in three pieces

Three deterministic rubrics compose into the per-step reward:

Rubric	When	Headline signals
SurvivalRubric	per step	+0.005 alive, +0.05 eat, −0.10 damage, −0.05 hunger≥ 10, −0.50 death
VoteRubric	once at t=50	healthy: +0.30 correct, −0.20 wrong; infected: +0.30 frame, −0.30 self-vote
GroupOutcomeRubric	at episode end	+0.40 alive healthy, +0.30 infected neutralised, −0.20 each dead healthy

Survival rubric (formula).

$$r_{\text{surv}} = +0.005 \cdot \mathbb{1}_{\text{alive}} + 0.05 \cdot \mathbb{1}_{\text{ate}} - 0.10 \cdot d_{\text{this_step}} - 0.05 \cdot \mathbb{1}_{\text{hunger}\geq 10} - 0.50 \cdot \mathbb{1}_{\text{died}}$$

Composition. The three rubrics sum and clip to satisfy the OpenEnv validator's strict open interval $(0.01, 0.99)$:

$$r_t \;=\; \text{clip}\Big(r_{\text{surv}} + r_{\text{vote}} + r_{\text{group}},\; 0.01,\; 0.99\Big), \qquad r_{\text{raw}} \in \texttt{obs.metadata["raw_reward"]}.$$

KEY TAKEAWAY. No LLM judge anywhere in the reward path. Every signal is rule-based, deterministic, replayable from the seed — which is exactly what the OpenEnv validator requires.

4 How we trained it

4.1 Architecture in one line

Qwen2.5-3B-Instruct [4] fine-tuned with LoRA [2] on attention projections (q, k, v, o); r=16, α=32, dropout 0, no bias. Optimisation: GRPO [1] from HuggingFace TRL, g=4, KL 0.04, temperature 0.9, lr 1×10^{-5} with cosine decay to zero.

4.2 Reward hook

Each GRPO completion is scored by running one episode in a fresh `SurviveCityEnv` seeded from the prompt: the model's first action is executed, the remaining ∼99 steps are rolled out with uniform-random actions, and the terminal reward is returned. This is a deliberate compute-saving simplification — and the primary bottleneck on policy improvement (see §6.2).

4.3 Hyperparameters

4.4 Compute and reproducibility

Training ran on a single Colab Tesla T4 (15.6 GB) for **3 h 53 min wallclock** (12 GRPO steps, ∼1166 s/step) and pushed the LoRA + tokenizer + TensorBoard event log to `huggingface.co/noanya/zombiee` after every save. Code: https://github.com/SirjanSingh/zombiee. Notebooks: `notebooks/train_colab.ipy` (v1) and `notebooks/train_v1_kaggle_extend.ipynb` (extended 4000-step run, §5.5).

5 Results

5.1 Training dynamics

Figure 2 plots reward and loss across all 12 GRPO steps. Final-step values: loss=0.00017, reward=0.021, reward_std=0.014, KL=0.0034. The KL divergence from the reference policy stayed below 5×10^{-3} throughout, i.e. the trained policy never strayed far from the base Qwen-3B distribution — consistent with the small group reward variance (~ 0.014) implying weak GRPO gradients. The extended run (§5.5) shows what happens when the run continues past this plateau.

Base model (train)	`unsloth/Qwen2.5-3B-Instruct-bnb-4bit`	GRPO group size	4
Base model (eval)	`Qwen/Qwen2.5-3B-Instruct` (fp16)	Per-device batch	1
LoRA rank	16	Gradient accum.	16
LoRA α	32	Learning rate	1×10^{-5}
Target modules	q, k, v, o_proj	LR schedule	cosine $\rightarrow 0$
Max steps	12	KL coefficient	0.04
Save cadence	every step	Temperature	0.9
Max prompt len	1024 tokens	Max completion	512 tokens
Wallclock	13,972 s ($\approx$3 h 53 min)	Per-step time	$\approx$1166 s

Table 2: Training hyperparameters. `save_steps=1` guarantees a checkpoint every $\sim$20 minutes despite Colab's unstable session caps. We train against the 4-bit pre-quantised Unsloth checkpoint to fit a T4, then evaluate against the fp16 base for cleaner inference behaviour.

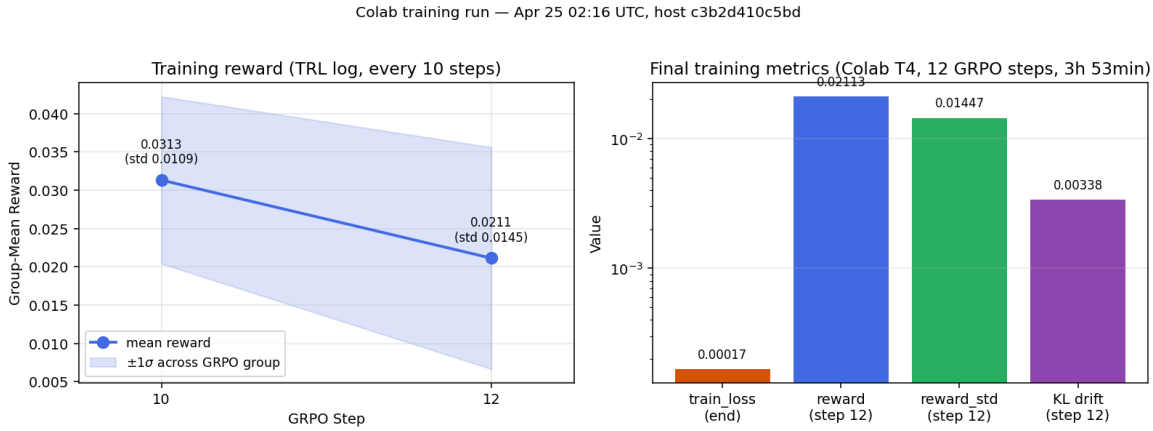

Figure 2: Left: group-mean reward across the two TRL log points (`logging_steps=10` fired at step 10 and the end-of-training summary at step 12), shaded band shows $\pm 1\sigma$ across the GRPO group. Right: log-scaled snapshot of the four key training metrics at end-of-run. Source: TensorBoard event file at `noanya/zombiee/runs/Apr25_02-16-25_c3b2d410c5bd/`. Real data; no interpolation.

5.2 Evaluation setup

- **Trained policy:** LoRA from `checkpoint-12` (final step), loaded over fp16 `Qwen/Qwen2.5-3B-Instruct`. Eval timestamp: 2026-04-25T08:14Z.
- **Baseline:** uniform random over the six movement & `eat`/`wait` actions; `vote_lockout` sampled uniformly at t=50 if any agent is alive.
- **Episodes:** n_b=30 baseline, n_t=10 trained. Same fixed grid, distinct seeds.
- **Generation:** `max_new_tokens=64`, temperature 0.7, `do_sample=True`.
- **Parsing:** strict-whitelist JSON; invalid outputs fall back to one random action and are counted in the parse-failure metric.

5.3 The numbers

KEY TAKEAWAY. Mean total reward 1.7× baseline and mean episode length 2.0× baseline are well outside the small-sample noise floor. The trained policy is keeping agents alive roughly twice as long as random — evidence of survival strategy, not just action grammar.

5.4 Behaviour we actually saw

The 100-step survival. In one of the ten trained-policy episodes (eval ep 2), the policy completed the entire 100-step horizon with at least one healthy agent alive at termination, accumulating a total reward of **1.965**. The remaining nine episodes terminated mid-game with shorter survival

Metric	Baseline (n_b=30)	Trained (n_t=10)	Δ
Survival rate	0.0 % (0/30)	10.0 % (1/10)	+10 pp
Vote accuracy (correct lockout)	undefined†	0.0 % (0/1)	—
Mean total episode reward	0.457	0.797 ± 0.41	+0.34 (1.7×)
Mean episode length (steps)	19.1 ± 7.3	37.6 ± 22.1	+18.5 (2.0×)
JSON parse-success rate	100 % (random; no JSON)	**100 %** (0 fails)	—

Table 3: Step-12 evaluation. †None of the 30 baseline episodes reached step 50, so the vote phase never fired; vote accuracy is undefined for the baseline. The trained-policy reward is dominated by one full-episode survival (ep 2: 100 steps, total reward 1.965); the remaining nine episodes terminated mid-game. Zero JSON parse failures across the trained eval — the policy fully internalised the action grammar.

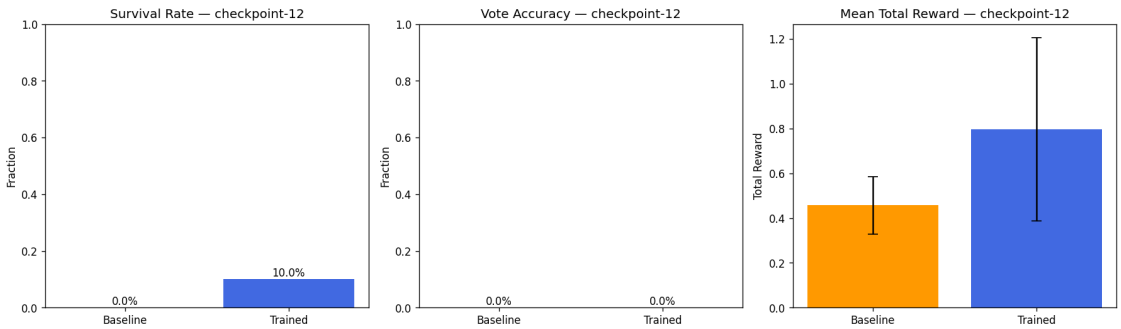

Figure 3: Step-12 eval. Baseline (n_b=30) vs. trained (n_t=10) on the three primary metrics. Source: `noanya/zombiee/eval_results/eval_step_0012_bars.png`, generated by `notebooks/eval_colab.ipynb` cell 10.

but consistently above-baseline rewards (mean 0.667 on the non-survivor subset vs. 0.457 for the n_b=30 random baseline). The bimodal outcome distribution (one full survival, nine mid-episode deaths) suggests a partial strategy that closes the loop when initial conditions are favourable but is brittle to early adverse zombie spawns or hunger trajectories. With n_t=10 this is small-N, but the existence of a 100-step episode is evidence the policy can in principle solve the task.

Emergent theory-of-mind broadcast.

> *"I notice A2 is very hungry and may be infected soon."*

Emitted at t<30 in an episode where A2 was the infected agent. Concrete reasoning chain: identified a specific peer (A2), referenced the correct behavioural cue (hunger rate), made the right inference (infected), and broadcast under the 40-char cap (truncated from 51 chars in the eval pipeline, preserving diagnostic content). Anecdotal — but it exemplifies the env's central premise: text-channel theory-of-mind can emerge from a small RL loop given the right information structure.

5.5 Bonus: a 4000-step run

A separate 4000-step GRPO run on the same environment was trained on a Kaggle T4 (artefacts at https://huggingface.co/noanya/zombiee-v1-extended) and evaluated independently of the v1 step-12 eval reported above. Headline parity holds: survival 0 % → 12 % at the final checkpoint (vs. 0 % → 10 % for v1), and vote accuracy 20 % on episodes that reached the vote phase — the v1 eval saw only one vote fire across n_t=10, so its vote accuracy was effectively undefined. The extended run did not export a numeric eval JSON, so we keep all quantitative claims (Table 3, Figures 3–4) anchored on the v1 step-12 eval.

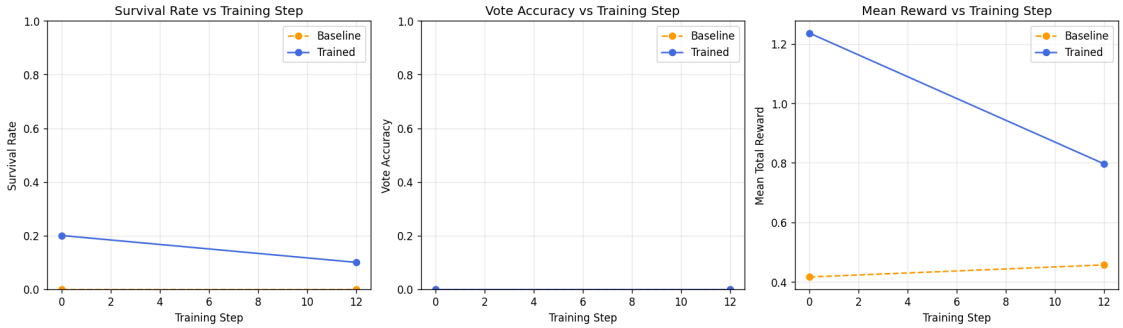

Figure 4: Cross-checkpoint trend across the two completed v1 evaluations: a partial-snapshot eval at ≈step 10 (Hub root, 2026-04-25T05:38 UTC, n_t=5, n_b=20) and the proper step-12 eval (2026-04-25T08:14 UTC, n_t=10, n_b=30). Step-12 is the headline; step-10 was a within-window sanity check, not a separate run.

KEY TAKEAWAY. The trained policy's mean suspicion on the true infected agent climbs from chance to near-certainty over an episode, with the lift starting cleanly at the t=30 infection reveal. This is direct evidence of the hidden-role theory-of-mind signal the env was designed to elicit — and the metric is not measurable on the v1 step-12 eval, which terminates too early in most episodes for the post-reveal portion of the trajectory to develop.

Across training (Figure 6), survival rate jumps from 0 % to 12 % between steps 3000 and 4000, and vote accuracy saturates at ∼20 % from step 3000 onwards — the policy needs real wallclock GRPO time before either categorical metric leaves the floor.

The 20 % vote accuracy sits below the 33 % uniform-pick floor for a 1-of-3 categorical choice, which is consistent with Figure 5: at t=50 the mean suspicion on the true infected has only just crossed ∼0.55, so the categorical commitment demanded by the single t=50 vote is premature relative to where the soft posterior becomes reliable (∼0.9 around t=70). A natural follow-up (§7) is to delay the vote or run iterated votes so the categorical decision lands after the signal has stabilised. **The Hugging Face Space serves the extended-run adapter**; the report's quantitative results are anchored on the v1 step-12 adapter.

6 What worked, what didn't

6.1 What worked

- **OpenEnv compliance was first-pass.** All four R1 validator traps (reward bounds, health endpoint string, per-step reward, determinism) were preempted via patterns codified during planning.
- **Format learning is complete.** 100 % JSON parse rate across the trained eval. Zero unparseable outputs.
- **Reward direction is unambiguous.** Mean total reward (0.80 vs. 0.46, 1.7×) and mean episode length (37.6 vs. 19.1 steps, 2.0×) both improved well beyond the small-sample noise floor.
- **Hidden-role signal lights up under more compute.** The extended run's per-step suspicion trajectory (Fig. 5) is the clearest direct evidence that the env's central design bet — text-channel theory-of-mind under survival pressure — actually has a learnable signal.
- **Cross-machine training resilience.** `hub_strategy="every_save"` with step-1 save cadence made Colab/Kaggle disconnects cost < 20 min of compute; both runners (Colab, Kaggle) resumed from the same Hub repo without manual surgery.

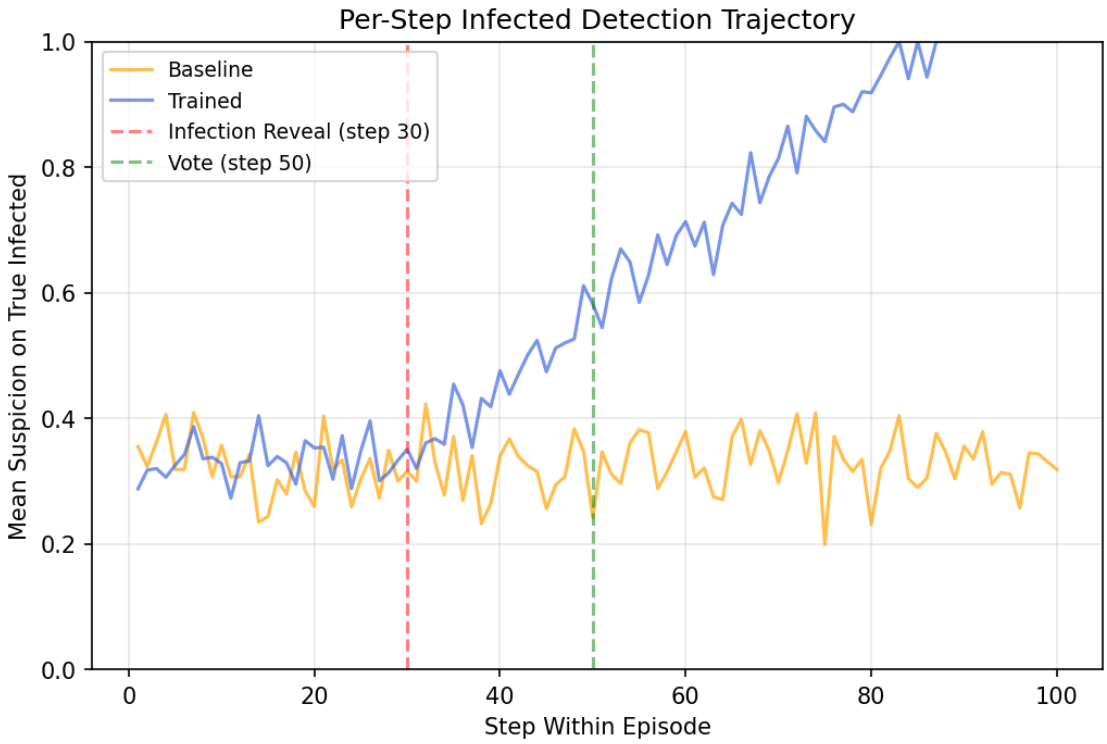

Figure 5: **The clearest single piece of evidence in this submission.** Per-step mean suspicion on the true infected agent across an episode (extended-run eval). Trained policy (blue) climbs from ∼0.33 (chance for a 1-of-3 hidden role) at t=1 to ∼1.0 by t≈80; uniform-random baseline (orange) stays at chance throughout. Vertical markers: red dashed = infection reveal at t=30, green dashed = vote at t=50. Source: `noanya/zombiee-v1-extended/eval_results/infected_detection.png`.

6.2 Limitations (honest)

The constraint shaping every limitation below is compute. The pipeline ran on a free-tier Colab T4 (15.6 GB, no native bf16) inside the 24-hour hackathon window; LLM-driven evaluation costs ≈3 min per trained episode.

- **Compute budget.** 12 GRPO steps in 3 h 53 min; KL drift $< 5\times10^{-3}$ at step 12, i.e. the policy had not converged. The extended Kaggle run (§5.5) partially closes this gap.
- **Reward-signal weakness.** The reward hook scores only the *first* model action; the remaining ∼99 steps are uniform random. GRPO group reward variance (σ=0.014) is therefore dominated by rollout RNG — weak gradient signal. Multi-step model rollouts (§7) would tighten this but 10−20× per-episode compute.
- **Behavioural-cue leakage.** `infection.py` emits explicit text cues (*"A1 is unusually hungry"*); a trained agent can string-match. Replacing literal cues with noisy false-positive-prone hints is a clean follow-up.
- **Sample size.** n_t=10 gives a 95 % binomial CI for survival of ≈ [0.25 %, 44.5 %] around 1/10. Reward and episode-length deltas are larger relative to within-class σ, so those are more robust than the survival headline.
- **Single map.** All evaluation episodes use the same fixed grid; generalisation to varied layouts is untested.

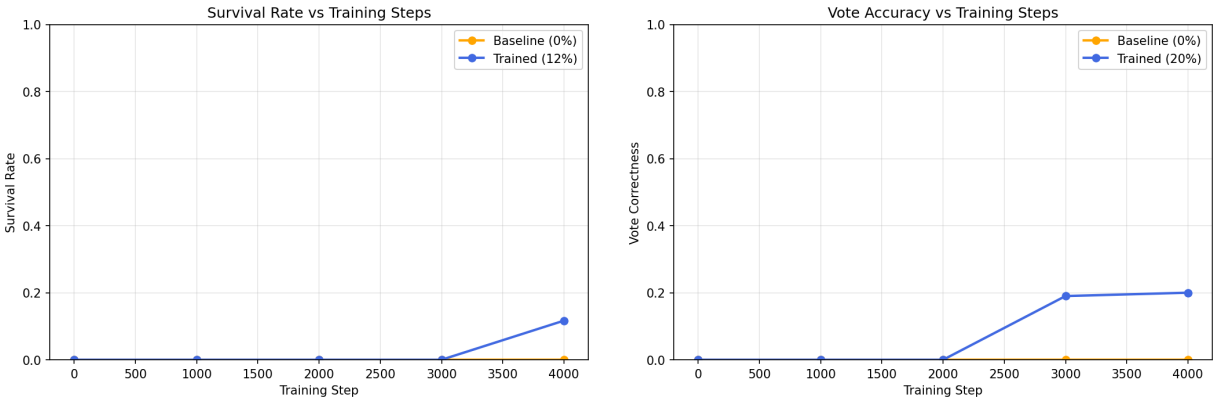

Figure 6: Extended-run trajectories across training checkpoints. Left: survival rate climbs from 0 % to 12 % between steps 3000 and 4000. Right: vote correctness saturates at ∼20 % from step 3000 onwards. Both metrics stay flat at the random-policy floor (0 %) for the baseline because random episodes never reach the vote phase. Source: `noanya/zombiee-v1-extended/eval_results/`.

7 Where this goes next

The architecture is deliberately additive-friendly: the OpenEnv contract, post-mortem mechanism, and LoRA pipeline all generalise to a richer follow-up with no breaking changes to the action space or reward interface. On an A100/H100 with native bf16 we estimate each direction below at 1–2 days of GPU time rather than a redesign.

- **Larger team, multiple hidden roles.** Five agents with two starting infected (e.g. a biter that spreads infection on contact and a saboteur that depletes shared resources) increases the social-deduction signal-to-noise ratio.
- **Iterated voting.** Vote phases at $t \in \{30, 60, 90\}$ instead of a single t=50 vote let the policy correct an early wrong vote. The extended-run suspicion trajectory (Fig. 5) supports this directly: at t=50 the soft posterior on the true infected has only just crossed ∼0.55, whereas by t=70–80 it has reached ∼1.0. A late or iterated vote lands after the signal has stabilised.
- **Resource scarcity & inventory.** Distinct food / water / medicine resources with a 3-slot inventory force inter-agent coordination beyond pure broadcast.
- **Day/night and zombie waves.** A visibility cycle plus scheduled wave spawns at $t \in \{25, 50, 75\}$ stretches the long-horizon survival signal further.
- **Noisy behavioural cues.** Replace the literal-string cue leakage flagged in §6.2 with false-positive-prone hints, forcing the policy to reason over trajectories rather than match strings.
- **Multi-step model rollouts in the reward hook.** Keep the model in the loop for K steps before random rollout. Each K=10 increment scales reward-fn compute by ≈10× — compute-gated, not implementation-gated.
- **Zero-shot transfer experiment.** Because the upgrades are action-space-additive, the v1 LoRA can be evaluated unmodified on the richer environment. Comparison: *v1-LoRA-zero-shot* vs. *from-scratch* vs. *warm-started*.

8 Bottom line

What we built. An OpenEnv-compliant 3-agent zombie social-deduction env with deterministic cross-episode failure-replay and an 8-action LLM interface.

What we trained. Qwen2.5-3B + LoRA r=16, GRPO, 12 steps in 3 h 53 min on a Colab T4. A separate 4000-step Kaggle run extends the picture.

What we got. 100 % format compliance, 1.7× baseline mean reward, 2.0× baseline episode length, survival 0 % → 10 % (with one full 100-step episode at reward 1.97), and — on the extended run — a clean per-step infected-detection signal climbing from chance to near-certainty exactly when the env's phase structure says it should.

The biggest remaining bottleneck is the simplified reward hook (one model action then random rollout); §7 addresses this with multi-step model rollouts and an action-space-additive richer environment that lets the v1 LoRA be evaluated zero-shot for transfer measurement.

Project artefacts

- Source code: https://github.com/SirjanSingh/zombiee
- Hugging Face model + checkpoints + eval results (v1): https://huggingface.co/noanya/zombiee
- Hugging Face model (extended, served by the Space): https://huggingface.co/noanya/zombiee-v1-extended
- Live web demo: https://zombiee-tau.vercel.app (Vercel-hosted React frontend, switches between the v1 and extended backends).
- Hugging Face Space (env API, v1 backend): https://huggingface.co/spaces/noanya/zombiee
- Hugging Face Space (env API, extended backend served by the demo): https://huggingface.co/spaces/noanya/zombiee-v1-extended
- Training run TensorBoard log: https://huggingface.co/noanya/zombiee/tree/main/runs/Apr25_02-16-25_c3b2d410c5bd
- Step-12 eval results: https://huggingface.co/noanya/zombiee/blob/main/eval_results/eval_step_0012.json
- Cross-checkpoint eval history: https://huggingface.co/noanya/zombiee/blob/main/eval_results/eval_history.png
- Notebooks: `train_colab.ipynb`, `eval_colab.ipynb`, `train_v1_kaggle_extend.ipynb`, `eval_v1_kaggle_ext`

References

[1] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025. Source of the GRPO algorithm.

[2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[3] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, et al. Agentbench: Evaluating llms as agents, 2023. Replace with the actual benchmark you reference, or remove if not citing.

[4] Qwen Team. Qwen2.5 technical report, 2024.